Jorg Brown <jorg.brown@gmail.com>
2019-06-17
Document D1715R0 $Id: proposal.html,v 1.50 2019/06/17 04:37:32 jorg Exp $

D1715 (Revision 0)

Revision history

- 2019-06-17: R0: Proposal to loosen restrictions on "_t" typedefs.

Introduction.

In C++14, templated typedefs and values were added alongside similarly-named templated classes. For example:

```
// C++11:
// "If B is true, the member typedef type shall equal T. If B is false, the member typedef type shall equal F."
template<bool b, class T, class F>
struct conditional;

// C++14
template<bool b, class T, class F>
using conditional_t = typename conditional<b, T, F>::type;
```

Unfortunately, this is all the standard has to say about conditional_t. In particular, the standard does not leave implementors the option of defining conditional in terms of conditional_t.

Motivation and Scope.

This may not seem to be important, however this definition of conditional_t requires that the compiler instantiate the templated class conditional for each combination of b/T/F template parameters, which turns out to be substantial, especially in environments where template meta-programming is in heavy use.

Consider an alternative implementation of conditional_t:

```
template<bool _Bp> struct __select;
template<> struct __select<true>  { template<typename _TrueT, typename _FalseT> using type = _TrueT;};
template<> struct __select<false> { template<typename _TrueT, typename _FalseT> using type = _FalseT;};

template <bool _Bp, class _TrueT, class _FalseT> using conditional_t = typename __select<_Bp>::template type<_TrueT, _FalseT>;
```

In this implementation, no matter how many times conditional_t is used, there are only two types that are instantiated as a result: __select<true> and __select<false>.

This not only saves in compiler memory, but also debug information. Here at Google, for a particularly TMP-heavy file, it turned out that around 1/6th of all classes emitted as part of clang's debug information were instantiations of std::conditional.

While proposing a change to fix this, and thus reduce debug size, it was pointed out that this change is actually observable. Suppose a function is declared in a header like this:

```
template<bool B> long to_long(conditional_t<B, int, long> param);
```

and then later in the same file, implemented like this, presumably because the programmer accidentally updated only one of two mentions of to_long:

```
template<bool B> long to_long(typename conditional<B, int, long>::type param) {
  return param;
}
```

The header compiles just fine but if conditional_t has not been defined in terms of conditional, then it turns out we have defined two different overloads. Then when to_long is called, we will get a "call is ambiguous" error. (See https://godbolt.org/z/tEH-pq)

Despite this theoretical impact, the actual impact is quite low; thankfully programmers almost never declare a function in terms of one templated type, and then define it with another.

Impact on the Standard

This proposal suggests that the while the type produced by the templated name_t variants of templated name_t classes must be identical, they are free to arrive at that type in any way the library author chooses.

Proposed Wording

Note: All changes are relative to the 2019-06-13 working draft of C++20.

Modify 20.5.2 [tuple.syn] :

```
template<size_t I, class T>
  using tuple_element_t = /* produces same type as typename tuple_element<I, T>::type */;
```

Modify 20.7.2 [variant.syn] :

```
template<size_t I, class T>
  using variant_alternative_t = /* produces same type as */typename variant_alternative<I, T>::type */;
```

Modify 20.14.1 [functional.syn] :

```
template<class T> using unwrap_ref_decay_t = /* produces same type as */typename unwrap_ref_decay<T>::type */;
```

Modify 20.15.2 [meta.type.synop] :

```
template<class T>
  using remove_const_t    = /* produces same type as */typename remove_const<T>::type */;
template<class T>
  using remove_volatile_t = /* produces same type as */typename remove_volatile<T>::type */;
template<class T>
  using remove_cv_t       = /* produces same type as */typename remove_cv<T>::type */;
template<class T>
  using add_const_t       = /* produces same type as */typename add_const<T>::type */;
template<class T>
  using add_volatile_t    = /* produces same type as */typename add_volatile<T>::type */;
template<class T>
  using add_cv_t          = /* produces same type as */typename add_cv<T>::type */;

template<class T>
  using remove_reference_t     = /* produces same type as */typename remove_reference<T>::type */;
template<class T>
  using add_lvalue_reference_t = /* produces same type as */typename add_lvalue_reference<T>::type */;
template<class T>
  using add_rvalue_reference_t = /* produces same type as */typename add_rvalue_reference<T>::type */;

template<class T>
  using make_signed_t   = /* produces same type as */typename make_signed<T>::type */;
template<class T>
  using make_unsigned_t = /* produces same type as */typename make_unsigned<T>::type */;

template<class T>
  using remove_extent_t      = /* produces same type as */typename remove_extent<T>::type */;
template<class T>
  using remove_all_extents_t = /* produces same type as */typename remove_all_extents<T>::type */;

template<class T>
  using remove_pointer_t = /* produces same type as */typename remove_pointer<T>::type */;
template<class T>
  using add_pointer_t    = /* produces same type as */typename add_pointer<T>::type */;

template<class T>
  using type_identity_t    = /* produces same type as */typename type_identity<T>::type */;
template<size_t Len, size_t Align = default-alignment> // see [meta.trans.other]
  using aligned_storage_t  = /* produces same type as */typename aligned_storage<Len, Align>::type */;
template<size_t Len, class... Types>
  using aligned_union_t    = /* produces same type as */typename aligned_union<Len, Types...>::type */;
template<class T>
  using remove_cvref_t     = /* produces same type as */typename remove_cvref<T>::type */;
template<class T>
  using decay_t            = /* produces same type as */typename decay<T>::type */;
template<bool b, class T = void>
  using enable_if_t        = /* produces same type as */typename enable_if<b, T>::type */;
template<bool b, class T, class F>
  using conditional_t      = /* produces same type as */typename conditional<b, T, F>::type */;
template<class... T>
  using common_type_t      = /* produces same type as */typename common_type<T...>::type */;
template<class... T>
  using common_reference_t = /* produces same type as */typename common_reference<T...>::type */;
template<class T>
  using underlying_type_t  = /* produces same type as */typename underlying_type<T>::type */;
template<class Fn, class... ArgTypes>
  using invoke_result_t    = /* produces same type as */typename invoke_result<Fn, ArgTypes...>::type */;
```

Last words

The author is unaware of observable impact of changing the "_v" constexpr definitions, such as tuple_size_v, to be computed in some other way, so no proposal is made to change them.